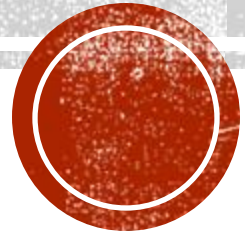
CLOUD COMPUTING

Chandara Chea

chandara.chea@cadt.edu.kh



SYLLABUS

1. Introduction to Cloud Computing
2. Cloud Computing Technologies
3. Networking in Cloud
4. Storage Solutions in the Cloud
5. Cloud Security and Compliance
6. Infrastructure as Code
7. Cloud Native
8. DevOps

WEEK 7 – CLOUD NATIVE

- **Learning Objectives:**
  - Understand what is cloud native application and its benefits
  - Understand core pillars of cloud native application
  - Understand what is monolith and its challenges
  - Understand what is microservices and its challenges
  - Understand how microservices communicates
  - Understand when to use and not to use microservices

WHAT IS CLOUD NATIVE?

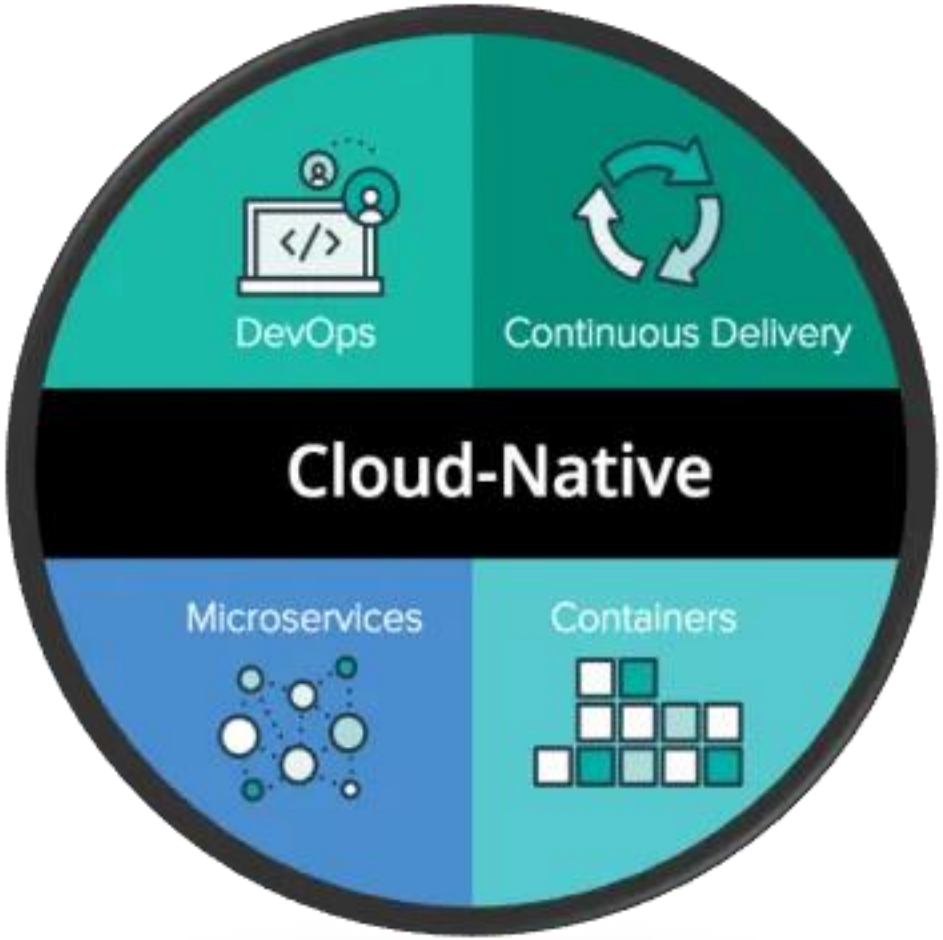
- AWS: Cloud native is the software approach of building, deploying, and managing modern applications in cloud computing environments.
- Google Cloud : Cloud native means adapting to the many new possibilities—but very different set of architectural constraints—offered by the cloud compared to traditional on-premises infrastructure.
- Microsoft : *Cloud-native architecture and technologies are an approach to designing, constructing, and operating workloads that are built in the cloud and take full advantage of the cloud computing model.*
- Cloud Native Computing Foundation : *Cloud-native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.*
- ***Cloud native is a blueprint for building web-scale applications on the cloud that are more available and scalable.***

BENEFIT OF CLOUD NATIVE APPLICATION

- Faster development: Developers use the cloud-native approach to reduce development time and achieve better quality applications. Instead of relying on specific hardware infrastructure, developers build ready-to-deploy containerized applications with DevOps practices. This allows developers to respond to changes quickly. For example, they can make several daily updates without shutting down the app.
- Platform independence: By building and deploying applications in the cloud, developers are assured of the consistency and reliability of the operating environment. They don't have to worry about hardware incompatibility because the cloud provider takes care of it. Therefore, developers can focus on delivering values in the app instead of setting up the underlying infrastructure.
- Cost-effective operations: You only pay for the resources your application actually uses. For example, if your user traffic spikes only during certain times of the year, you pay additional charges only for that time period. You do not have to provision extra resources that sit idle for most of the year.

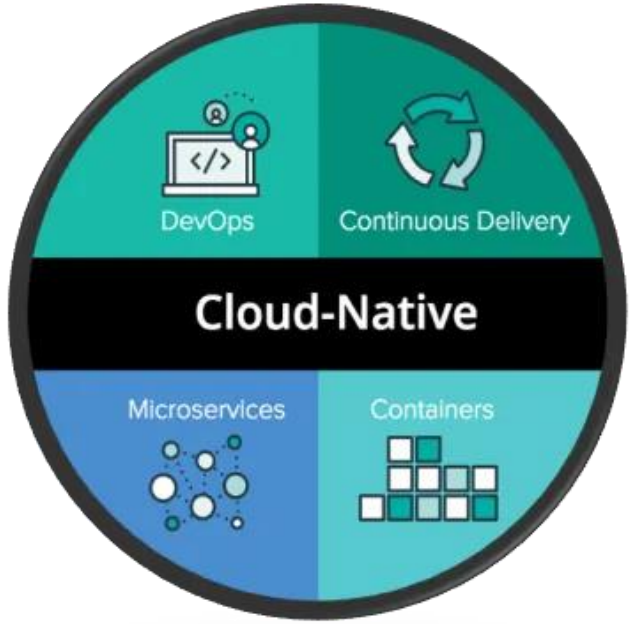
CLOUD NATIVE

- **DevOps** is the collaboration between software developers and IT operations with the goal of automating the process of software delivery & infrastructure changes.
- **Continuous Delivery** enables applications to released quickly, reliably & frequently, with less risk.
- **Micro-services** is an architectural approach to building an application as a collection of small independent services that run on their own and communicate over HTTP APIs.
- **Containers** provide light-weight virtualisation by dynamically dividing a single server into one or more isolated containers.



WHAT IS DEVOPS?

- **DevOps** is a set of practices, principles, and cultural philosophies that aim to bridge the gap between **software development (Dev)** and **IT operations (Ops)** teams. The goal is to improve collaboration, communication, and automation in the process of software delivery and infrastructure management. This enables organizations to develop, test, and release software more rapidly, efficiently, and with higher quality.



KEY CONCEPTS OF DEVOPS

- **Collaboration & Communication**: DevOps fosters a culture where developers, operations staff, QA teams, and other stakeholders work together more closely. This helps break down silos between different departments.
- **Automation**: Automation is at the heart of DevOps. It involves automating repetitive tasks like deployment, configuration management, testing, and monitoring to reduce human error, increase speed, and enhance reliability.
- **Continuous Integration (CI)**: Developers frequently integrate their code changes into a shared repository. Automated tests are run to catch bugs early, ensuring that changes are always compatible with the existing codebase.
- **Continuous Delivery/Continuous Deployment (CD)**: CI often leads to Continuous Delivery, where code changes are automatically built, tested, and prepared for release into production. Continuous Deployment takes this a step further, automatically deploying code to production without manual intervention.
- **Monitoring & Feedback**: Continuous monitoring of applications and infrastructure allows teams to identify issues proactively, understand user experiences, and iteratively improve the system. Feedback loops are critical for ongoing optimization.
- **Microservices Architecture**: Many DevOps teams embrace a microservices architecture, which breaks applications into small, manageable components that can be developed, tested, and deployed independently.

WHAT IS CI/CD?

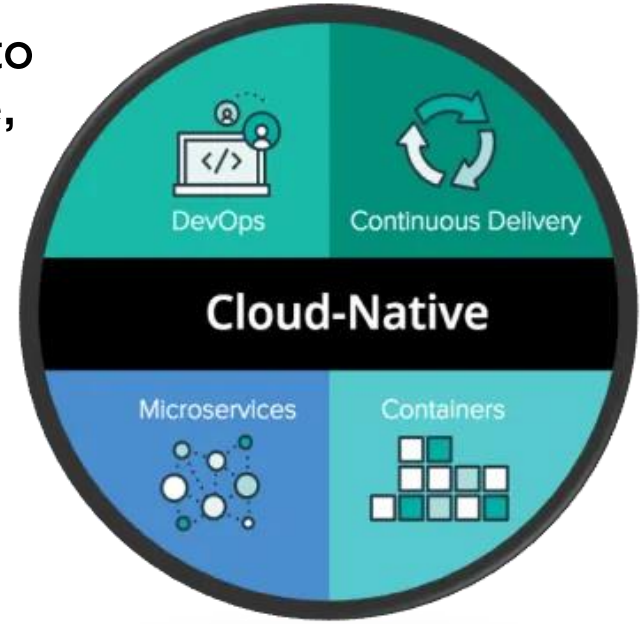
- **CI/CD** stands for **Continuous Integration** and **Continuous Delivery** (or **Continuous Deployment**), and it represents a set of modern software development practices and principles aimed at improving the software development lifecycle. CI/CD practices help teams automate and streamline the process of integrating code changes, testing, and deploying software.
- **Continuous Integration** refers to the practice of frequently integrating code changes into a shared repository (often multiple times a day). The key goals of CI are to detect issues early, improve code quality, and ensure that code is always in a deployable state.
  - **Frequent code commits**: Developers commit small changes to a shared codebase often (sometimes multiple times per day).
  - **Automated builds**: Each commit triggers an automatic build process. This ensures that the changes integrate correctly and that the system is always in a working state.
  - **Automated testing**: Automated tests (unit, integration, etc.) are run with every integration to detect bugs and regressions early.
  - **Fast feedback**: Developers get quick feedback on their code changes (whether they passed or failed tests), allowing them to fix issues quickly before they compound.
  - **Example of CI tools**: Jenkins, CircleCI, Travis CI, GitLab CI, GitHub Actions.

CONTINUOUS DELIVERY

- **Continuous Delivery** extends the concept of CI by automatically preparing code changes for production deployment. In Continuous Delivery, the code is always in a deployable state, and it can be released to production at any time, although the actual release is still a manual decision.
  - **Automated deployment pipelines**: Code changes automatically go through a pipeline that includes building, testing, and staging environments.
  - **Frequent releases**: Since code is always in a deployable state, releases to production can happen quickly and more often (but are still triggered manually or through approvals).
  - **Staging environments**: Code is deployed to a staging environment that mirrors production before being released, ensuring the release process is smooth.
  - **Example of Continuous Delivery tools**: Jenkins, CircleCI, GitLab CI, Spinnaker, ArgoCD.

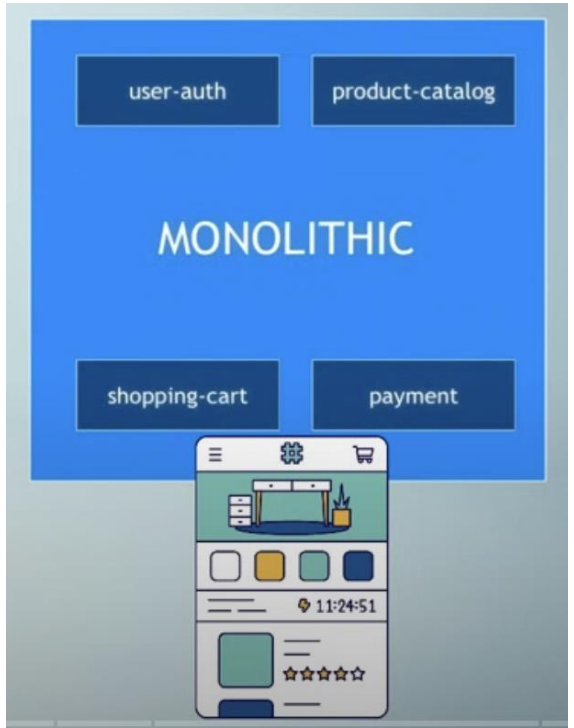
CONTAINERS

- **Containerization** is a technology that allows developers to package applications and their dependencies into isolated units called **containers**.
- A **container** is a lightweight, standalone, and executable package that contains everything needed to run a piece of software: the application code, runtime, system tools, libraries, and dependencies.
- A **container image** is a read-only template used to create containers. It includes the application code, libraries, and environment variables required to run the application.



MONOLITH ARCHITECTURE

- **Monolithic architecture** is a traditional approach to software design where an entire application is built as a single, unified unit. In a monolithic system, all components—such as the user interface (UI), business logic, database access, and other services—are tightly integrated and packaged together into one large, self-contained application.
- All components are part of a single unit.
- Everything is developed, deployed and scaled as 1 unit
- Application is written with 1 tech stack
- Teams need to be careful not to affect each other's work
- Redeploy the entire application on each update

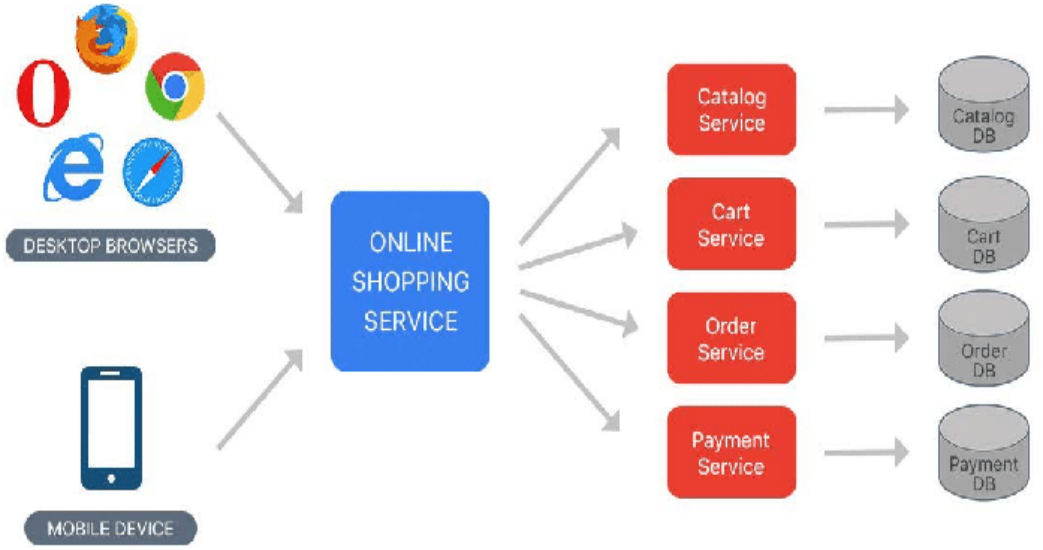


CHALLENGES OF MONOLITH

- Application is too large and complex
- You can only scale entire application instead of specific service
- Difficulty if services need different dependency versions
- Release process take longer
- On every change, the entire application needs to be tested, built and deployed
- Bug in any module can bring down the whole application

MICROSERVICES ARCHITECTURE

- **Microservices architecture** is a style of software development in which an application is broken down into a collection of **small, independent services** that each perform a specific, narrowly defined function. Each microservice operates as an isolated unit, communicates with other services via APIs (usually HTTP/REST or messaging queues), and can be developed, deployed, and scaled independently.
- Split based on business functionalities
- Separation of concern: 1 service for 1 specific job
- Self-contained and independent
- Developed, deployed and scaled separately
- Loosely coupled

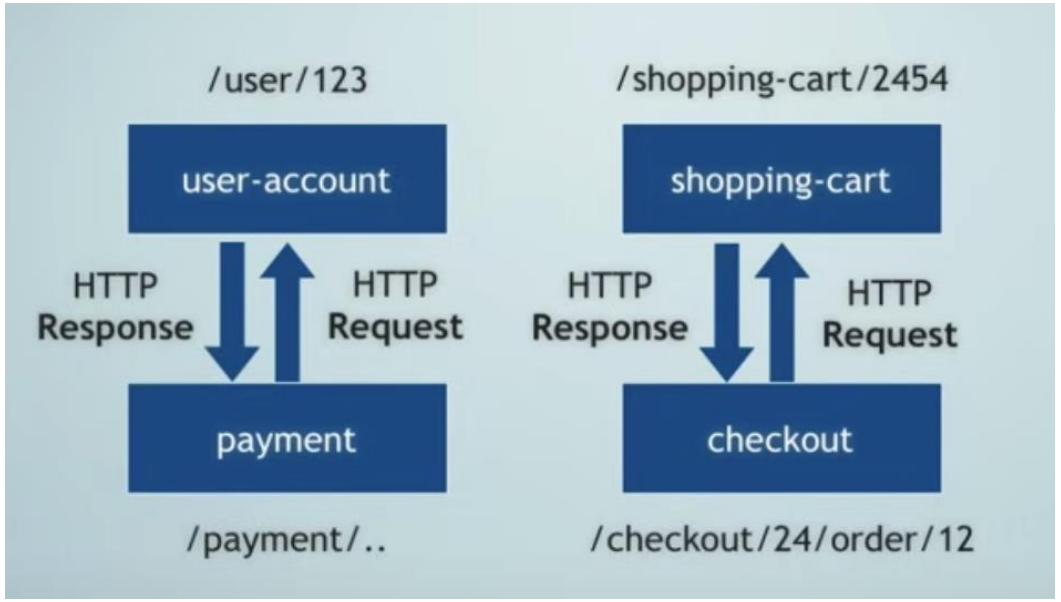


CHALLENGES OF MICROSERVICES

- Microservices introduce operational complexity. Managing multiple services, monitoring, and coordinating them can be difficult.
- Ensuring data consistency across multiple services can be challenging, especially when each service has its own database.
- The network calls between services can become a bottleneck, and handling failures in communication (timeouts, retries, etc.) becomes critical.
- Dealing with things like service discovery, load balancing, and fault tolerance can be more complicated than in a monolithic application.
- Managing deployments of numerous microservices, particularly in production, often requires sophisticated orchestration tools (like Kubernetes).

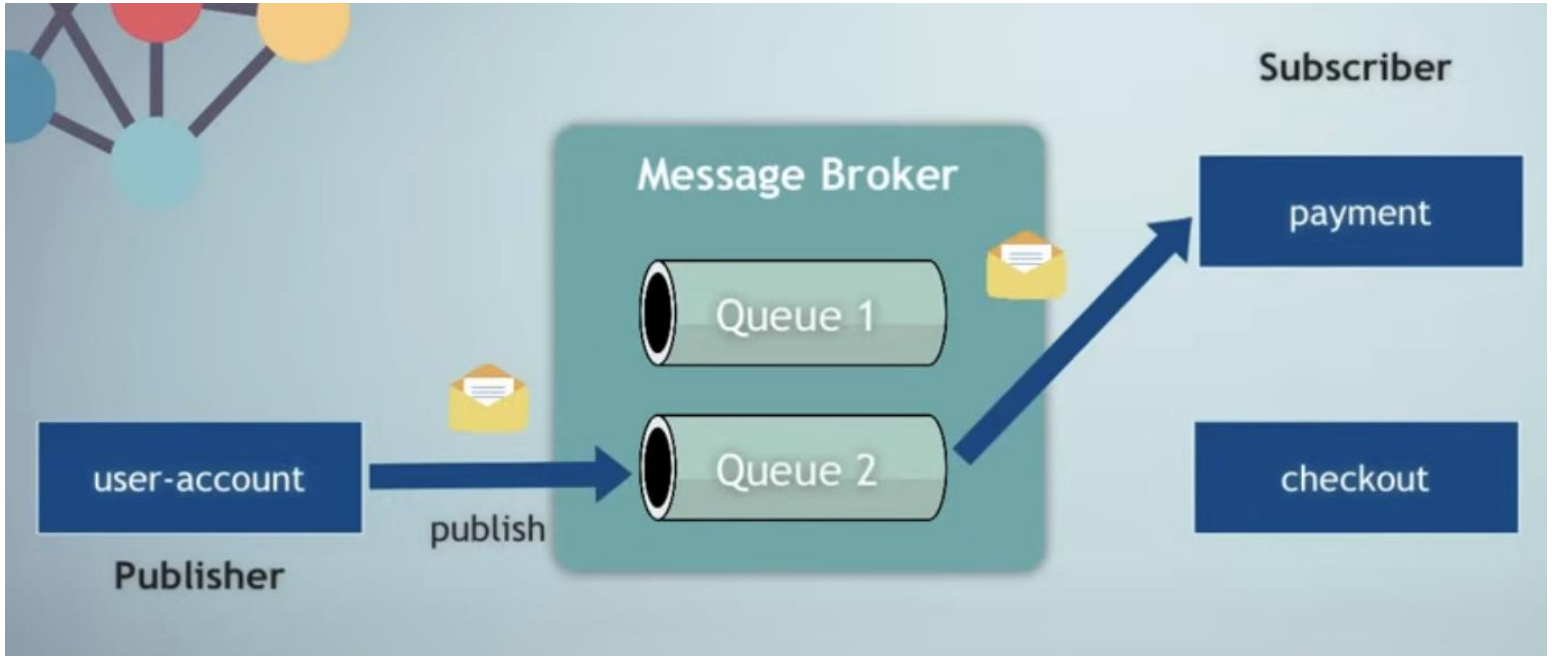
COMMUNICATION BETWEEN SERVICES

- Communication via API calls
  - Each service has its own API
  - They can talk to each other, by sending request to the respective API endpoint
  - Synchronous communication where one service sent request to another and wait for response



COMMUNICATION BETWEEN SERVICES

- Communication via message broker
  - Asynchronous communication
  - Publisher/subscriber model: publisher send message to intermediary service/message broker then message broker send message to subscriber



WHEN TO USE MICROSERVICES?

- your application is growing too large to maintain as a monolith
- your application needs to support multiple, diverse business domains
- different parts of your application need to be scaled independently
- you need frequent, independent releases of different parts of the system
- your development team is large and needs to be split into smaller, cross-functional teams
- you need improved fault tolerance
- you are adopting a cloud-native, containerized approach

WHEN NOT TO USE MICROSERVICES?

- Small Applications or Startups
- Limited Resources
- Tight Deadlines
- Lack of Need for Scalability